`constexpr` Function Parameters

Document Number: P1045R1
Date: 2019-09-27
Author: David Stone (david.stone@uber.com, david@doublewise.net)
Audience: Evolution Working Group (EWG)

Abstract

With this proposal, the following code would be valid:

```
void f(constexpr int x) {
    static_assert(x == 5);
}
```

The parameter is usable in all the same ways as any `constexpr` variable.

Moreover, this paper proposes the introduction of a "maybe constexpr" qualifier, with a strawman syntax of 🤯`constexpr`🤯 (this syntax is a placeholder for most of the paper, there is a section on syntax later on). Such a function can accept values that are or are not `constexpr` and maintain that status when passed on to another function. In other words, this is a way to deduce and forward `constexpr`, similar to what "forwarding references" / "universal references" (`T &&`) do in function templates today. This paper proposes adding generally usable functionality to test whether an expression is a constant expression (`is_constant_expression`), with the primary use case being to use this test in an `if constexpr` branch in such a function to add in a compile-time-only check.

Finally, this paper proposes allowing overloading on `constexpr` parameters. Whether a parameter is `constexpr` would be used as the final step in function overload resolution to resolve cases that would otherwise be ambiguous. Put another way, we first perform overload resolution on type, then on `constexpr`.

This paper has three primary design goals:

1. Eliminate arcane metaprogramming from modern libraries by allowing them to make use of "regular" programming.
2. Allow library authors to take advantage of known-at-compile-time values to improve the performance of their libraries
3. Allow library authors to take advantage of known-at-compile-time values to improve the diagnostics of their libraries

Changes In This Revision

R1: The original version proposed a particular model of `static` function variables. R1 discusses the problems in that original model under the section "Function static variables". R1 also adds more detail on how overload resolution is supposed to work under the section "Overload resolution". Propose a new syntax, including changing the meaning of `constexpr` on a variable declaration and supporting `consteval` variables.

No Shadow Worlds

Malte Skarupke wrote an article on language design and "shadow worlds". Shadow worlds are parts of a language which are segregated from the rest of the language. When you are stuck in this "shadow" language, you find yourself wanting to use the full power of the "real" language. C++ contains at least four general purpose languages (three of them shadow languages): "regular" C++, `constexpr` functions, templates, and macros.

Prior to C++14, constexpr functions couldn't have `if` or `for`, leading to contorted recursive code full of `return condition ? complicated_expression1 : complicated_expression2`. Since C++14, we cannot allocate memory in constexpr functions. Once we remove that restriction in C++20, we still cannot `reinterpret_cast`. The trend here is an ever increasing move toward making `constexpr` functions just be regular functions.

Templates are in an even worse spot: another Turing complete language that has no loops, no mutation, and a completely different syntax for everything.

Worse still are macros: no loops, mutation, recursion, scoping, no real functions, and weird if statements.

For this paper, the shadow world of templates is the most relevant. In a strong twist of irony, Malte's article talks about macros being a shadow world in every language, so therefore you want a strong template system that can replace macros. The template system, however, is yet another shadow language. In C++, as in other languages, the compile-time arguments are split from the run-time: we have template parameters and function parameters, so you can write f<0>(1). That seems fine, at first, until you realize that there are many things you cannot do because of that syntactic difference.

You cannot pass template arguments to constructors. You cannot pass template arguments to overloaded operators. You cannot overload on whether a value is known at compile time when they have different syntaxes. You cannot put a compile-time argument after a run-time argument, even if that is where it makes the most sense for it to be. You cannot generically forward a pack of arguments, some of them compile time and some of them run time. All of these limitations lead to us reimplementing basic functionality like loops, partial function application, and sorting for "run time" vs "compile time" values.

These problems motivate this paper. By treating compile-time and run-time values more uniformly, we bring these two worlds together. Large portions of existing metaprogramming libraries going back to Boost.MPL try to work around the problems, but we need a language change to solve it at the root.

Before and After

Now (without this proposal)	**The future (with this proposal)**

`auto a = std::array<int, 2>{};`
`a[0] = 1;`
`a[1] = 5;`
`a[2] = 3; // undefined behavior`
 	
`auto t = std::tuple<int, std::string>{};`
`std::get<0>(t) = 1;`
`std::get<1>(t) = "asdf";`
`std::get<2>(t) = 3; // compile error`
	`auto a = std::array<int, 2>{};`
	`a[0] = 1;`
	`a[1] = 5;`
	`a[2] = 3; // compile failure`
	
	`auto t = std::tuple<int, std::string>{};`
	`t[0] = 1;`
	`t[1] = "asdf";`
	`t[2] = 3; // compile failure`

`std::true_type{}`	`true`

`std::integral_constant<int, 24>{}`
`std::constant<int, 24> // proposed`
`std::constant<24> // discussed`
`boost::hana::int_c<24>`
`boost::mpl::int_<24>`
	`24`

`template<int n>` `void f(boost::hana::int_<n>);`	`f(constexpr int x);`

`0 <=> 1 == 0;`
`// valid, as intended`
 	
`0 <=> 1 == nullptr;`
`// valid, due to implementation detail`
	`0 <=> 1 == 0;`
	`// valid, as intended`
	
	`0 <=> 1 == nullptr;`
	`// error: no overloaded operator== comparing`
	`// strong_equality with nullptr_t`

`// Either all regex constructions have an`
`// extra pass over the input to determine the`
`// parsing strategy, or...`
`auto const r = std::regex("(A+|B+)*C");`
`// exponential time against failed matches`
`// starting with 'A' and 'B'.`
`// https://swtch.com/~rsc/regexp/regexp1.html`
	`// Scan can be done at compile time, so...`
	
	`auto const r = std::regex("(A+|B+)*C");`
	`// linear time against failed matches`
	`// starting with 'A' and 'B'.`

`auto const glob = std::regex("*");`
`// throws std::regex_error`
	`auto const glob = std::regex("*");`
	`// static_assert failed "Regular expression`
	`// token '*' must occur after a character to`
	`// repeat."`

`static_assert(`
`  pow<3>(2_meters) == 8_cubic_meters`
`);`
`meters runtime_value;`
`std::cin >> runtime_value;`
`auto const computed = pow<2>(runtime_value);`
`if (computed > 100_square_meters) {`
`  throw std::exception{};`
`}`
	`static_assert(`
	`  pow(2_meters, 3) == 8_cubic_meters`
	`);`
	`meters runtime_value;`
	`std::cin >> runtime_value;`
	`auto const computed = pow(runtime_value, 2);`
	`if (computed > 100_square_meters) {`
	`  throw std::exception{};`
	`}`

`// Do I want my code to work at compile time,`
`// do I want my code to be fast, or do I try`
`// to modify my compiler to recognize the`
`// code and generate the right assembly at`
`// run time when it is written in the`
`// `constexpr` style?`
	`// From the std-proposals forum`
	`size_t strlen(constexpr char const * s) {`
	`  for (const char *p = s; ; ++p) {`
	`  if (*p == '\0') {`
	`  return p - s;`
	`  }`
	`  }`
	`}`
	`std::size_t strlen(char const * s) {`
	`  __asm__("SSE 4.2 insanity");`
	`}`

`// Cannot write a function that transparently`
`// uses certain intrinsics where possible.`
`// Cannot write a function that puts the`
`// parameters in the correct order without`
`// resorting to `integral_constant` business.`
	`#include <emmintrin.h>`
	
	`void do_math(__v2di x, constexpr char z) {`
	`  __v2di y;`
	`  // ... some code ...`
	`  // This intrinsic requires the third`
	`  // argument to be a compile-time constant`
	`  x = __builtin_ia32_pclmulqdq128(x, y, z);`

```
    // ...
}
```

Background

This proposal will reference a few libraries that make heavy use of compile-time parameters. If readers are not familiar with these libraries, that is fine, the below should be sufficient description to understand some of the motivation of this paper. This paper will also reference other papers that are currently under discussion.

Boost.MPL

The Boost.MPL library is a general-purpose template metaprogramming framework of compile-time algorithms, sequences, and metafunctions. It was designed under the constraints of C++03. MPL organizes itself around passing types to class template parameters. It includes things like `int_` as a way to turn a value into a type so that values can be used with the type-based parts of the library.

Boost.Hana

Boost.Hana is a metaprogramming library that aims to make metaprogramming just "programming", using as much "regular" C++ as possible. It does this by merging types and values -- values are encoded in types, and types are turned into values -- so that most of the user interaction with `boost::hana` is by calling what appear to be regular functions.

bounded::integer

`bounded::integer` is a library that adds compile-time range checking to integer types. `bounded::integer<5, 1000>` is an integer type that can be between 5 and 1000, inclusive, and the resulting type of `bounded::integer<1, 10> + bounded::integer<2, 5>` is `bounded::integer<3, 15>`.

Class Types in Non-Type Template Parameters

C++20 allows class types as non-type template parameters. To use such a type, it must be a literal type with a defaulted `operator==`, with all bases and data members meeting the same requirement ("strong structural equality").

Why we need this proposal

`constexpr` parameters have several advantages over existing solutions:

- Better user experience due to uniform syntax on the caller's side: They do not need to remember `f(true_)` or `f(constant<true>)` for `f<true>()` or or anything else like that. The standard and most intuitive way to pass a value to a function is as a function parameter: `f(true)`.
- Better compile times as a result of being able to just pass things like `true` rather than instantiating a wrapper template. This applies primarily to solutions like `boost::mpl` (where everything has to be a type, including integers, and is the origin of `integral_constant`) and `boost::hana` (where everything is passed as a regular function parameter, even compile-time constants, which uses the equivalent of `integral_constant` but passed by value instead of by type). The cost of time and memory to compile `void f(constexpr int x)` should be essentially the same as `template<int x> void f()`.
- Library authors can add compile-time diagnostics without adding run-time code generation for checking. This is somewhat possible now, but it requires that the caller put the result in a constexpr context and does not work in all cases (for instance, `std::regex` cannot be a literal type due to dependence on `std::locale`). There is a large amount of user confusion today when a `constexpr` function is passed all `constexpr` parameters, but an error is caught at run time instead of compile time because the result of that function was not stored in a `constexpr` result.
- Allows adding a new overload that can take advantage of known-at-compile-time values, without forcing all callers to change
- Allows easier integration of strongly-typed code with existing code. There are many generic libraries that expect to be able to initialize integer types with a literal `0` or `1`. To support this, the `bounded::integer` library would have to accept all arguments of type `int` (and hope that the library never passes `0U`), which removes some of the compile-time safety from the library. If `bounded::integer` could know that it was dealing with a safe value, it would go a long way toward improving interoperability.

How does this work

The model that this proposal tries to follow is that the behavior of a `constexpr` function parameter exactly matches the behavior of a non-type template parameter as much as is possible. More specifically, the expected implementation of this feature would be to have the equivalent of a template instantiation for each value of a `constexpr` parameter used by the program. Using a `constexpr` function parameter creates a new template instantiation for each distinct constant expression value, plus a single instantiation for all run-time values (if the function is ever called with a run-time value).

In which contexts can `constexpr` parameters be used?

This paper proposes allowing a `constexpr` parameter anywhere that any other `constexpr` value could be used in any location that appears lexically after the parameter. In this way, a `constexpr` parameter is usable in the same way as a template parameter. In particular, the following code is valid:

```
auto f(constexpr int x, std::array<int, x> const & a)
    noexcept(x > 5)
    requires(x % 2 == 0)
{
    static_assert(x == 6);
    return std::array<int, x + 9>();
}
```

Function static variables

There are two possible mental models for this feature. This paper proposes that it is an alternative syntax for templates, but another plausible mental model is that these are regular functions for which some parameters are available for use at compile time. Those two models are not in conflict for most cases, but when `static` variables come into play, they suggest very different results.

```
int & f(constexpr int) {
    static int x = 0;
    return x;
}
++f(0);
std::cout << f(1);
```

What is this program guaranteed to print? The "actually a template" model says it should print "0" because these are two different functions. The "regular function" model says it should print "1" because the variable x is declared `static`, and since we have written just one function here, we are using the same variable. One of primary goals of this proposal is to allow a more regular style of programming, rather than separate rules for run time vs. compile time. Therefore, this paper has a goal of aligning with the "regular function" model. Unfortunately, the rules for templates are the way they are for a reason. There are many examples that seem to suggest that the template model is the only reasonable model:

```
void f(constexpr int n) {
    static constexpr int x = n;
    static_assert(x == n);
}
```

Is this `static_assert` guaranteed to succeed? With the regular function model, this function would not be guaranteed to succeed, because the first invokation of the function determines the value of x. However, there is no fixed ordering of "first" at compile time, especially considering that the function can be called from two different translation units with two different arguments. This suggests that it is not workable to have static constexpr variables of dependent value in such functions. We run into the same problems with static variables of dependent type.

```
void f(constexpr int n) {
    using T = std::conditional_t<n == 0, int, long>;
    static T x;
}
```

The definition of "dependent type" ends up applying to any locally defined type.

```
void f(constexpr int n) {
    static struct {
        static int f() {
            return n;
        }
    } s;
}
```

This may not seem like a very common problem (locally defined structs are uncommon in most code bases), but we do have one common example of local types:

```
auto f(constexpr int n) {
    static auto foo = [] { return n; };
}
```

Lambdas create a unique type (and here we must create a unique type for the code to make any sense).

It also feels like there is a required symmetry between the identity of static variables and the identity of types returned. It is a must-have that the type returned from such a function can be dependent on the value for it to solve the use cases outlined at the start of the paper. This essentially gives us only two options: either we have the same behavior as templates do (there is a separate static variable per constexpr value) or we would have to not allow static variables (possibly just no static variables of dependent type or value). I am neutral on which option we choose.

Overload resolution

Determining which function is called is divided into two steps: determining viable candidates, and then determining the best match from that set of viable candidates. `constexpr` parameters will change both of these.

Viable candidates

Determining viable candidates follows a simple rule that should mostly match intuition: if a function parameter is marked `constexpr`, the corresponding argument must be able to initialize a constexpr variable of that same type. Examples:

```
void a(constexpr int x); // 1
void a(int x); // 2

a(1); // 1 and 2 are viable

int x = 5;
a(x); // only 2 is viable


void b(constexpr bool x);
void b(bool x);

std::true_type x;
b(x); // 1 and 2 are viable due to constexpr conversion operator
```

Overload resolution

Overload resolution is a bit more complicated, but once again has a fairly simple intuitive backing. The general idea is that `constexpr` is a tie-breaker for otherwise equivalent overloads. This is similar to what we do for top-level cv-qualifiers on references: "S1 and S2 include reference bindings ([dcl.init.ref]), and the types to which the references refer are the same type except for top-level cv-qualifiers, and the type to which the reference initialized by S2 refers is more cv-qualified than the type to which the reference initialized by S1 refers." This feature would be specified similarly, but talk about `constexpr` instead of references and cv-qualifiers.

```
void f0(int);
void f0(constexpr long);
f0(42); // calls f0(int)

void f1(int);
void f1(constexpr int);
f1(42); // calls f1(constexpr int)

void f2(unsigned);
void f2(constexpr long);
f2(42); // ambiguous
```

What are the restrictions on types that can be used as `constexpr` parameters?

There are two possible options here. Conceptually, we would like to be able to use any literal type as a `constexpr` parameter. This gives us the most flexibility and feels most like a complete feature. Again, however, the obvious implementation strategy for this paper would be to reuse the machinery that handles templates now: internally treat `void f(constexpr int x)` much like `template<int x> void f()`. In the "template" model, we would limit `constexpr` parameters to be types which can be passed as template parameters: strong structural equality. This paper proposes following the lead of non-type template parameters.

Can you take the address of `constexpr` parameter?

Just like in P0732, we would like to be able to call const member functions on these parameters at run time, but for that to be possible they need to have an address. To satisfy that requirement, non-type template parameters of class type are const objects, of which there is one instance in the program for every distinct constexpr parameter value. P0732 proposes this only for non-type template parameters of class type because built-in non-type template parameters are already defined as being prvalues. For constexpr function parameters, these feels like a strange difference. Any solution here ends up leading to inconsistencies with other parts of the language, and thus this section needs further thought.

Function Pointers

This paper does not propose adding support for pointers to functions accepting `constexpr` parameters simply because the full interactions there have not yet been explored. Theoretically this could be supported if the type of the function pointer was `void(constexpr int)` and the function pointer is formed in a constexpr context. There appears to be some precedent in this space if we were to consider the function pointer type to have a `consteval` copy constructor, but this is not being proposed at this time.

Perfect forwarding

It would be nice to be able to write a function template that accepts parameters that might be `constexpr` and forward them on to another function. For example, if I have a class that wraps another type, it would be nice to write a constructor that forwards all arguments on to the wrapped type. To do this, we need some sort of perfect forwarding mechanism. There are three possibilities here:

1. Do not solve this problem. Ignoring this problem causes an exponential explosion on forwarding functions and makes it impossible to write variadic versions of such functions.
2. Follow the lead of rvalue vs. lvalue references and implement a "constexpr deduction" rule. In other words, for `template<typename T> void f(T x)`, the type `T` can be deduced as `constexpr int`. This seems appealing at first, because of the analogy to rvalue references and the fact that existing code would automatically become "aware" of this feature and take advantage of it. However, that is both a benefit and a drawback, because it means that all function templates get this behavior. This could have a very large cost in time and memory at compile time if compilers have to activate much of their current template machinery to implement this. This also requires using a deduced type even when the exact type is known. In other words, all constexpr values passed to existing templates suddenly instantiates different templates where before they instantiated the same template.
3. "Maybe constexpr" parameters. `void f(🤯constexpr🤯 int x)` is a (strawman) syntax to allow users to write a single function overload that can accept parameters that can be used as compile-time constants and parameters that cannot.

Option 3 seems like the best option, but requires some more explanation for how it would work. There are two primary use cases for this functionality. The first use case is to forward an argument to another function that has a 🤯`constexpr`🤯 parameter or is overloaded on `constexpr`. The second use case is to write a single function that has one small bit of code that is run only at compile time or only at run time. For instance, `std::array::operator[]` could be implemented like this:

```
auto & array::operator[](🤯constexpr🤯 size_t index) {
    if constexpr (is_constant_expression(index)) {
        static_assert(index < size());
    }
    return __data[index];
}
```

We put in a compile-time check where possible, but otherwise the behavior is the same. I actually expect this to be the primary way to overload `constexpr` parameters with run-time parameters when all that is needed is a minor difference, as it ensures there are no accidental differences in the definition.

This is a sample implementation of `is_constant_expression`, defined as a macro, which is possible to write in standard C++ with the introduction of this paper:

```
std::true_type is_constant_expression_impl(constexpr auto);
std::false_type is_constant_expression_impl(auto);

#define is_constant_expression(...) \
    decltype(is_constant_expression_impl( \
        (void(__VA_ARGS__), 0) \
    ))::value
```

This must be a macro so that it does not evaluate the expression and perform side effects. We need to make it unevaluated, and the macro wraps that logic. I believe it shows the strength of this solution that users can use it to portably detect whether an expression is constexpr (a common user request for a feature), rather than needing that as a built-in language feature. If we had lazy function parameters, the implementation would be even more straightforward:

```
constexpr bool is_constant_expression(~LAZY~ constexpr auto) { return true; }
constexpr bool is_constant_expression(~LAZY~ auto) { return false; }
```

Effect on the standard library

This paper is expected to have a fairly small impact on the standard library, because we currently make use of few non-type template parameters on function templates. The feature will obviously add a new tool for libraries to begin taking advantage of, but I don't believe there are any changes which would be embarrassing or otherwise mandatory to make in response to this feature.

Related work

It is recommended to go through the list of examples at the beginning and think about how they could be implemented using any of the features listed below. Some of the examples can be implemented in a way that is just as straightforward, but most of them cannot be implemented at all with any other proposed feature. The following features are commonly compared to constexpr function parameters.

`std::is_constant_evaluated()`

`std::is_constant_evaluated()` allows the user to test whether the current evaluation is in a context that requires constant evaluation, so the user could say `if (is_constant_evaluated()) {} else {}`. This solves only some of the problems addressed by this paper. The general issue is that

`std::is_constant_evaluated()` is testing how the expression is used, but `constexpr` function parameters are all about what data is provided. To clarify the difference, consider an alternative universe where we want to restrict memory orders to be compile-time constants, rather than the current state where they could be run-time values. With constexpr parameters, the solution is straightforward:

```
void atomic::store(T desired, constexpr memory_order order = memory_order_seq_cst) noexcept;
```

This would require the user to pass the memory order as a compile-time constant, but places no constraints on the overall execution being constexpr (and in fact, we know it never will be because atomic is not a literal type). It is not possible to implement this with `std::is_constant_evaluated()`. For a perhaps more compelling example, this is how we likely would have originally designed tuple if we had constexpr parameters:

```
constexpr auto && tuple::operator[](constexpr std::size_t index) { ... }
```

where the tuple variable (`*this`) is (potentially) a run time value. In other words, the code using the index is mixed in with the code using the tuple. `std::is_constant_evaluated()` allows you to write different code depending on whether the entire evaluation is part of a constexpr context. `constexpr` parameters allows you to partially apply certain arguments at compile time without needing to compute the entire expression at compile time.

Another important difference is that `if constexpr (std::is_constant_evaluated())` is meaningless: `if constexpr` is a context that requires a constant expression, and thus `std::is_constant_evaluated()` will always return true and the branch will always be taken. Many of the examples rely on being able to, for instance, return different types or write code in different branches that would not compile if the value is not a constant expression.

`consteval`

This has many of the same limitations as `std::is_constant_evaluated()`. It also does not provide a way to overload on "known at compile time" vs. "not known at compile time".

Parametric expressions: P1121

P1121 introduces the concept of "hygienic macros"

```
using add(auto a, auto b) { return a + b; }
```

It supports evaluating exactly once or 0-N times. It proposes allowing constexpr parameters, but just to the parametric expressions. Parametric expressions create yet another shadow world. In other words, if we were to accept P1121, we would have a new type of function that can be used to solve most of the problems solved by this paper, but only if you can also work with the other differences between parametric expressions and normal functions (no overloading is the most prominent example). It does not aim to completely replace existing functions, and thus would be an incomplete solution to this problem.

Syntax / Bikeshedding

There are two language-level features being proposed by this paper: 1) allowing function parameters to be annotated in some way to allow them to be used at compile time within the function and require initialization from a constant expression, and 2) allowing function parameters to be annotated in some way to allow them to be used at compile time if they are initialized with a constant expression. This section will discuss the syntax for those two features. All of this paper other than this section will use `constexpr` with its current meaning and 🤯`constexpr`🤯 for "maybe constexpr" parameters to avoid confusion.

`constexpr` and `consteval` today:

	variables	functions
(nothing)	must be done at run time	must be done at run time
constexpr	must be done at compile time	done at compile time if necessary, usable at compile time if possible
consteval	not supported	must be done at compile time

Option 1

	variables	functions
(nothing)	must be done at run time	must be done at run time
constexpr	done at compile time if necessary, usable at compile time if possible	done at compile time if necessary, usable at compile time if possible
consteval	must be done at compile time	must be done at compile time

Which simplifies to

(nothing)	definitely run time

constexpr	run time or compile time
consteval	definitely compile time

There is currently an inconsistency in the meaning of `constexpr`. On a variable declaration, it means "must be evaluated at compile time", but on a function declaration, it means "can be evaluated at compile time if it only does stuff that can be done at compile time". In other words, on a function, `constexpr` today means "maybe constexpr". We recently added a new keyword, `consteval`, that can only be applied to function declarations and it means "must be evaluated at compile time". We could take this as an opportunity to remove this inconsistency. This option would be to expand the keyword `consteval` to be usable on variable declarations and require compile-time evaluation of the variable. `constexpr` on a function declaration would retain its current meaning, but `constexpr` on a variable declaration would mean that the variable can be used in a constant expression context if it was initialized with a constant expression. The suggested meaning of the keywords would be that `consteval` means the thing is evaluated at compile time and `constexpr` means the thing is evaluated at compile time if the expression it uses can be evaluated at compile time.

This has the advantage of removing an existing inconsistency in the meaning of the `constexpr` keyword. It is also a backward-compatible change. It has the downside that the intent of users writing `constexpr` today could be to require a diagnostic if code ever changes that does not provide a compile-time constant. There could be a migration path, however.

C++17

```
// must be initialized at compile time
constexpr int x = foo();
```

C++20 (proposed)

```
// constinit is redundant
constinit constexpr int x = foo();
```

C++23 (proposed)

```
// constinit requires compile-time
constinit constexpr int x = foo();
```

Option 2

We keep things as they are today with regards to the meaning of `constexpr` and `consteval`. We allow annotating a function parameter with `constexpr` with the same meaning as a variable declaration: must be initialized with a constant expression. We add a new keyword, `maybe_constexpr`, that deduces whether the parameter is known at compile time. This paper originally proposed the syntax `constexpr?`. `consteval` was originally spelled as `constexpr!`, but `constexpr!` was rejected in favor of `consteval` for a variety of reasons, some of which also apply to `constexpr?`. One of the primary objections to names with punctuation is the question of how you pronounce the keyword. It would make spoken C++ be a tonal language, or else people will come up with a different name, such as "maybe constexpr". If people will come up with a different name for the keyword, we might as well name the keyword that name, thus `maybe_constexpr`.

FAQ

Default parameters

Q. Is the following code valid?

```
int runtime() { return 5; }
auto foo(constexpr int n = runtime()) -> std::array<int, n> { return {}; }
int main() {
    foo(1); // OK
    foo();  // ill-formed, presumably?
}
```

A. No. It is invalid in the first line, just like the following code is invalid today in all versions of C++:

```
int runtime() { return 5; }
template<int n = runtime()>
void foo();
```

The following similar code would be valid in the first call and invalid in the second call:

```
int runtime() { return 5; }
auto foo(🤯constexpr🤯 int n = runtime()) -> std::array<int, n> { return {}; }
int main() {
    foo(1); // OK
    foo();  // ill-formed because std::array<int, n> is ill-formed
}
```

Summary

The following represents a summary of all of the changes presented assuming we take my preferred direction for all choices offered by the paper. All of this is targeted at C++23 unless otherwise specified.

- Allow declaring a function parameter `consteval`, meaning that it can only be initialized as a compile-time constant and it is usable as a compile-time constant in the function.
- Declaring an automatic variable `consteval` means that it must be initialized at compile time and can only be used at compile time.
- Allow declaring a function parameter `constexpr`, meaning that if it is initialized as a compile-time constant, it is usable as a compile-time time constant in the function.
- Declaring an automatic variable `constexpr` means that it can be used at compile time if it is initialized at compile time, and it can always be used at run time.
- Allow overloading between `consteval` and unannotated parameters.
- Add a library function `std::is_constant_expression` that can be used on any expression and returns whether that expression is a constant expression.
- For C++20: Allow declaring a `constexpr` variable with the `constinit` keyword. It was not allowed simply because it is redundant under the current rules. Allowing this combination in C++20 gives users a migration path to maintain current behavior. This is unnecessary if we decide to not change the meaning of `constexpr` on automatic storage duration variables.
- For C++20: Adopt "Fixing inconsistencies between `constexpr` and `consteval` functions".